

Trabajo Final — Programación Lineal

Curso: Optimización (2026-1) · Universidad de Antioquia · Facultad de Ingeniería **Equipo:** Gabriel Cardona, Juan Sebastián Tabares, Víctor Rodas **Fecha de entrega:** 30 de mayo de 2026 **Repositorio del programa:** [proyectos/optimizacion-pl/](#)

1. Descripción del programa

El programa es un **solver interactivo de Programación Lineal (PL)** desarrollado en **Python 3.13** con interfaz web en **Streamlit**. Resuelve problemas de PL formulados en su **forma básica** (antes de la aumentada) y muestra la solución **paso a paso**, tablero por tablero, replicando el formato de las diapositivas del curso.

El usuario puede:

1. Definir un problema de PL con cualquier número de variables (2 a 8) y restricciones (1 a 10).
2. Especificar maximización o minimización.
3. Usar restricciones de tipo $\leq$, $\geq$ o $=$.
4. Cargar ejemplos predefinidos (Wyndor Glass, problema de la dieta, producción simple).
5. Ver la solución óptima, los tableros de cada iteración, el análisis de sensibilidad, el método gráfico (si hay 2 variables) y la versión matricial del Simplex Revisado.

El solver **detecta automáticamente** qué método aplicar:

- **Simplex tabular básico** si todas las restricciones son del tipo $\leq$ (la solución básica inicial trivial es factible).
- **Método de la Gran M** si hay restricciones $\geq$ o $=$ (se requieren variables artificiales).

Adicionalmente expone una vista del **Simplex Revisado (forma matricial)** y un **análisis de sensibilidad post-óptimo**.

2. Problema de referencia (modelo)

Como problema base se utilizó el clásico de **Wyndor Glass** (Hillier & Lieberman, *Introducción a la Investigación de Operaciones*), pero el programa es genérico:

Wyndor Glass debe decidir cuántos lotes producir de dos productos nuevos (puertas y ventanas) para maximizar la utilidad, sujeto a la disponibilidad de tiempo en tres plantas de producción.

Formulación:

$$\text{MAX } Z = 3 \cdot x_1 + 5 \cdot x_2$$

s.a.

$$\begin{array}{lll} x_1 & \leq 4 & \text{(Planta 1)} \\ & 2 \cdot x_2 \leq 12 & \text{(Planta 2)} \\ 3 \cdot x_1 + 2 \cdot x_2 & \leq 18 & \text{(Planta 3)} \\ x_1, x_2 & \geq 0 & \end{array}$$

Solución óptima (verificada): $x_1 = 2$, $x_2 = 6$, $Z^* = 36$.

Esta formulación permite además ilustrar el método gráfico ($n = 2$) y comparar contra las diapositivas y los textos clásicos. El programa también fue validado con un problema mixto ($\leq$, $=$, $\geq$) — el **problema de la dieta** — y con casos de infactibilidad y no acotación.

3. Arquitectura del programa

```
optimizacion-pl/
├── app.py                # UI Streamlit
├── requirements.txt
├── README.md
├── solver/
│   ├── problem.py       # LPPProblem, Constraint → conversión a form
│   ├── tableau.py       # Representación de un tablero Simplex
│   ├── simplex.py       # Simplex tabular básico
│   ├── big_m.py         # Método de la Gran M
│   ├── revised.py       # Simplex revisado (forma matricial)
│   ├── graphical.py     # Método gráfico (matplotlib)
│   └── sensitivity.py   # Análisis de sensibilidad
└── tests/
    ├── test_simplex.py
    └── test_all.py
```

Separación de responsabilidades: el módulo `solver/` es independiente de la UI y puede usarse desde cualquier interfaz (consola, otro framework, etc.).

4. Operaciones que realiza el programa

4.1 Lectura y normalización

El usuario ingresa el problema en **forma básica**:

- Vector de coeficientes objetivo $c = [c_1, c_2, \dots, c_n]$.
- Lista de restricciones, cada una con coeficientes a_{ij} , tipo ($\leq$, $\geq$, $=$) y lado derecho b_i .

El programa convierte automáticamente a **forma estándar**:

1. Si la función objetivo es de minimización, multiplica c por -1 (resuelve internamente como maximización).
2. Si algún $b_i < 0$, multiplica toda la fila por -1 e invierte el sentido de la desigualdad.
3. Por cada restricción agrega las variables auxiliares necesarias:
 - s_i (**holgura**) para $\leq$
 - e_i (**exceso**) y a_i (**artificial**) para $\geq$
 - a_i (**artificial**) para $=$
4. Construye la matriz A , el vector b y el vector c extendido.
5. Identifica la base inicial (las holguras y artificiales son básicas en el tablero inicial).

4.2 Método Simplex tabular

Si no hay variables artificiales, aplica el **algoritmo Simplex** clásico:

1. Construye el tablero $(m+1) \times (n+1)$ con $-c$ en la fila z y $[A \mid b]$ debajo.
2. En cada iteración:
 - Selecciona la **variable entrante** con la columna de coeficiente reducido más negativo (regla de Dantzig).
 - Calcula la **razón mínima** b_i / a_{ij} para filas con $a_{ij} > 0$.
 - Si todas las razones son no positivas $\rightarrow$ **solución NO ACOTADA**.
 - Si hay empate $\rightarrow$ **regla de Bland** (menor índice básico) para evitar ciclos.
 - Realiza el **pivoteo** (normaliza la fila pivote y elimina la columna pivote del resto).
3. Termina cuando todos los coeficientes reducidos son no negativos $\rightarrow$ **óptimo**.
4. Si en el óptimo existe alguna variable no básica con coeficiente reducido $= 0 \rightarrow$ **óptimos múltiples**.

Cada tablero se guarda con su número de iteración, fila/columna pivote, variable entrante y saliente, y se renderiza en la UI con el pivote resaltado.

4.3 Método de la Gran M

Para restricciones $\geq$ o $=$, se introducen variables artificiales a_i penalizadas con $-M$ (con $M = 10^6$) en la función objetivo. El procedimiento:

1. Construye el tablero inicial con la penalización.
2. **Limpia la fila z:** para cada artificial básica, resta $M \times \text{fila}_i$ de la fila z (de manera que el coeficiente reducido de las básicas iniciales sea 0).
3. Aplica el algoritmo Simplex igual que en 4.2.
4. Al terminar:
 - Si alguna artificial básica tiene valor $> 0 \rightarrow$ **solución INFECTIBLE**.
 - En caso contrario, retorna el óptimo en términos del problema original.

4.4 Método gráfico ($n = 2$)

Para problemas de dos variables:

1. Calcula los **vértices** de la región factible intersectando todos los pares de rectas (incluidos los ejes coordenados).
2. Descarta vértices que violan alguna restricción o tienen coordenadas negativas.
3. Evalúa Z en cada vértice y selecciona el óptimo según el sentido (max/min).
4. Grafica con `matplotlib`:
 - Recta de cada restricción.
 - Región factible sombreada (polígono).
 - Vértices con sus coordenadas.
 - Curva de nivel $Z = Z^*$ que pasa por el óptimo.
 - El vértice óptimo destacado.

4.5 Simplex Revisado (matricial)

Implementación equivalente al Simplex tabular pero expresada con operaciones matriciales explícitas. Para cada iteración se calcula y muestra:

- B : matriz base actual (columnas de A asociadas a la base).

- B^{-1} : inversa de la base.
- c_B : vector de coeficientes objetivo de las variables básicas.
- $x_B = B^{-1} \cdot b$: valores de las variables básicas.
- $y = c_B \cdot B^{-1}$: vector de **precios sombra** (variables duales).
- $r_j = c_j - y \cdot A_j$: costos reducidos de las variables no básicas.

La regla de selección de variable entrante (mayor r_j) y de variable saliente (razón mínima sobre $B^{-1} \cdot A_j$) es idéntica al Simplex tabular.

4.6 Análisis de sensibilidad

A partir del tablero final, el programa calcula:

- **Precios sombra (y)**: coeficiente de la holgura s_i en la fila z del tablero óptimo. Indica cuánto cambia Z^* por una unidad adicional del recurso b_i .
- **Costos reducidos**: coeficiente de cada variable no básica en la fila z. Indica cuánto debería mejorar c_j para que esa variable entre en la solución óptima.
- **Rangos de RHS (b_i)**: intervalo $[b_i + \Delta_{\min}, b_i + \Delta_{\max}]$ dentro del cual la base óptima se mantiene válida. Se calcula a partir de la inversa de la base B^{-1} y el vector x_B .
- **Rangos de coeficientes objetivo (c_j)**: intervalo dentro del cual la solución óptima actual sigue siendo óptima. Distingue entre variables básicas (análisis vía cambios en la fila z usando la fila pivote) y no básicas (cota superior dada por el costo reducido actual).

5. Casos especiales

El programa reconoce y reporta:

Caso	Detección
Óptimo único	Todos los coeficientes reducidos > 0 al converger.
Óptimos múltiples	Existe variable no básica con coeficiente reducido $= 0$ en el óptimo.
No acotado	La columna entrante no tiene coeficientes positivos para razón mínima.
Infactible	(Gran M) Alguna variable artificial queda con valor > 0 en el óptimo.
Degeneración	Empate en la razón mínima $\rightarrow$ se aplica regla de Bland para evitar ciclos.

6. Validación

Se validó contra los siguientes problemas con resultado conocido:

Problema	Esperado	Obtenido
Wyndor Glass (Hillier)	$Z = 36, x = (2, 6)$	✓
Dieta (mixto $\leq / = / \geq$)	$Z = 5.25, x = (7.5, 4.5)$	✓
Max $x_1 + x_2$ s.a. $-x_1 + x_2 \leq 1$	No acotado	✓
$x_1 + x_2 \leq 2 \wedge x_1 + x_2 \geq 5$	Infactible	✓
Precios sombra Wyndor	$(0, 1.5, 1)$	✓
Rango RHS Planta 2 Wyndor	$[6, 18]$	✓